



T.C.

NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

Yapay Zeka Teknikleri Dersi Projesi

Derin Sinir Ağları

Emre KARATOP - 220610006

1. Giriş

1.1. Problem Tanımı ve Amaç

Beton binalar, köprüler ve yollar zamanla yapısal bütünlüğü tehdit eden çatlaklar oluşturabilir. Bu çatlakların manuel olarak denetlenmesi (gözle kontrol) zaman alıcı, maliyetli ve insan hatasına açıktır.

Bu çalışmanın amacı; Convolutional Neural Networks (CNN) kullanarak, yapısal yüzey görüntülerini "Çatlak Var (Positive)" ve "Sağlam (Negative)" olarak sınıflandıran otomatik bir yapay zeka sistemi geliştirmektir. Çalışmada farklı derinlik, genişlik ve optimizasyon algoritmalarına sahip 4 farklı model mimarisi test edilerek en yüksek performansı veren yapı belirlenmeye çalışılmıştır.

2. Gelişme

2.1. Kullanılan Veri Seti

- Veri Seti: Kaggle Surface Crack Detection.
- Sınıflar: İkili sınıflandırma (Binary Classification) - Positive (Çatlaklı) / Negative (Sağlam).
- Görüntü İşleme: Tüm görüntüler 128x128 piksel boyutuna getirilmiştir.
- Piksel değerleri 0-255 aralığından 0-1 aralığına (Normalization) çekilmiştir.
- Veri seti %80 Eğitim (Training) ve %20 Doğrulama (Validation) olarak ayrılmıştır.

2.2. Kullanılan Yöntem ve Algoritmalar

Yapılan testlerde 4 farklı senaryo denenmiş ve aşağıdaki sonuçlar elde edilmiştir:

Senaryo 1: Baz Model (Reference Baseline)

- Mimari: 3 Konvolüsyon Katmanı (32, 64, 128 filtre) + 1 Dense Katman.
- Optimizasyon: Adam
- Sonuçlar:
 - Accuracy: %99.69
 - Loss: 0.0122
- Analiz: Bu model referans noktası olarak kurulmuştur. %99.69'luk başarı, veri setindeki çatlak özelliklerinin (kenar, kontür) oldukça belirgin olduğunu ve temel bir CNN yapısının bile bunu öğrenmekte zorlanmadığını gösterir. Adam optimizyer, hızlı bir yakınsama (convergence) sağlamıştır.

```
1 model_1 = Sequential([
2     # Giriş ve Ölçeklendirme
3     Rescaling(1./255, input_shape=(128, 128, 3)),
4
5     # 1. Blok (32 Filtre)
6     Conv2D(32, (3, 3), activation='relu'),
7     MaxPooling2D(2, 2),
8
9     # 2. Blok (64 Filtre)
10    Conv2D(64, (3, 3), activation='relu'),
11    MaxPooling2D(2, 2),
12
13    # 3. Blok (128 Filtre)
14    Conv2D(128, (3, 3), activation='relu'),
15    MaxPooling2D(2, 2),
16
17    # Sınıflandırma Kısmı
18    Flatten(),
19    Dense(128, activation='relu'),
20    Dense(1, activation='sigmoid') # Çıktı
21 ])
22
23 # Derleme (Adam)
24 model_1.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Senaryo 2: Derin ve Dar Mimari (Deep & Narrow)

- Mimari: 4 Konvolüsyon Katmanı (16, 32, 64, 128) + 2 Dense Katman.
- Optimizasyon: SGD (Stochastic Gradient Descent)
- Sonuçlar:
 - Accuracy: %99.77
 - Loss: 0.0092
- Analiz: Filtre sayısı azaltılıp (daraltılıp) katman sayısı artırılmıştır (derinleştirilmiştir). Derinliğin artması, modelin daha soyut özellikleri öğrenmesini sağlamıştır. SGD algoritması, Adam'a göre genellikle daha yavaş öğrenir ancak "Local Minima"dan (yerel minimum) kurtulma ve genelleme yeteneği daha yüksektir. Loss değerinin Baz modele göre düşmesi (0.0092), modelin tahminlerinden daha emin olduğunu gösterir.

```
1  model_2 = Sequential([
2      Rescaling(1./255, input_shape=(128, 128, 3)),
3
4      # 1. Blok (16 Filtre - Dar Başlangıç)
5      Conv2D(16, (3, 3), activation='relu'),
6      MaxPooling2D(2, 2),
7
8      # 2. Blok (32 Filtre)
9      Conv2D(32, (3, 3), activation='relu'),
10     MaxPooling2D(2, 2),
11
12     # 3. Blok (64 Filtre)
13     Conv2D(64, (3, 3), activation='relu'),
14     MaxPooling2D(2, 2),
15
16     # 4. Blok (128 Filtre - Derinlik Artışı)
17     Conv2D(128, (3, 3), activation='relu'),
18     MaxPooling2D(2, 2),
19
20     # Sınıflandırma Kısmı (Çift Dense)
21     Flatten(),
22     Dense(64, activation='relu'),
23     Dense(32, activation='relu'),
24     Dense(1, activation='sigmoid')
25 ])
26
27 # Derleme (SGD)
28 opt = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
29 model_2.compile(loss='binary_crossentropy', optimizer=opt, metrics=['accuracy'])
```

Senaryo 3: Geniş Ağ (Wide Network)

- Mimari: 3 Konvolüsyon Katmanı ancak yüksek filtre sayıları (64, 128, 256).
- Optimizasyon: RMSprop
- Sonuçlar:
 - Accuracy: %99.75
 - Loss: 0.0147
- Analiz: Modelin genişliği (kapasitesi) artırılmıştır. RMSprop kullanılmıştır. Doğruluk oranı yüksek olsa da, Loss değeri (0.0147) tüm senaryoların en yükseğidir. Bunun sebebi, çok geniş ağların veriyi ezberlemeye (overfitting) daha meyilli olması veya RMSprop'un bu veri setinde Adam kadar stabil çalışmaması olabilir. Model doğru bilse bile, hata payı (confidence margin) diğerlerine göre biraz daha yüksektir.

```
1  model_3 = Sequential([
2      Rescaling(1./255, input_shape=(128, 128, 3)),
3
4      # 1. Blok (64 Filtre - Geniş Başlangıç)
5      Conv2D(64, (3, 3), activation='relu'),
6      MaxPooling2D(2, 2),
7
8      # 2. Blok (128 Filtre)
9      Conv2D(128, (3, 3), activation='relu'),
10     MaxPooling2D(2, 2),
11
12     # 3. Blok (256 Filtre - Yüksek Kapasite)
13     Conv2D(256, (3, 3), activation='relu'),
14     MaxPooling2D(2, 2),
15
16     # Sınıflandırma Kısmı (Geniş Dense)
17     Flatten(),
18     Dense(256, activation='relu'),
19     Dense(1, activation='sigmoid')
20 ])
21
22 # Derleme (RMSprop)
23 model_3.compile(loss='binary_crossentropy', optimizer='rmsprop', metrics=['accuracy'])
```

Senaryo 4: Robust Model (Dropout Destekli)

- Mimari: 4 Konvolüsyon Katmanı (32, 64, 128, 256) + Dropout Regularization.
- Optimizasyon: Adam
- Sonuçlar:
 - Accuracy: %99.89
 - Loss: 0.0054
- Analiz: Bu senaryoda hem derinlik artırılmış hem de Dropout (Seyreltme) tekniği uygulanmıştır. Dropout, eğitim sırasında rastgele nöronları kapatarak modelin tek bir yola bağımlı kalmasını engeller ve ezberlemeyi önler.
 - Sonuç: En yüksek doğruluk (%99.89) ve en düşük hata payı (0.0054) bu modelde görülmüştür. Bu, modelin sadece çatlakları ezberlemediğini, çatlakların "ne olduğunu" gerçekten öğrendiğini ve en kararlı (stable) yapı olduğunu kanıtlar.

```
1 model_4 = Sequential([
2     Rescaling(1./255, input_shape=(128, 128, 3)),
3
4     # 1. Blok
5     Conv2D(32, (3, 3), activation='relu'),
6     MaxPooling2D(2, 2),
7     # Dropout(0.25) -> Genelde ilk katmana konmaz ama opsiyoneldir.
8
9     # 2. Blok
10    Conv2D(64, (3, 3), activation='relu'),
11    MaxPooling2D(2, 2),
12    Dropout(0.25), # %25 Unutma
13
14    # 3. Blok
15    Conv2D(128, (3, 3), activation='relu'),
16    MaxPooling2D(2, 2),
17    Dropout(0.25), # %25 Unutma
18
19    # 4. Blok (En derin özellikler)
20    Conv2D(256, (3, 3), activation='relu'),
21    MaxPooling2D(2, 2),
22    Dropout(0.25),
23
24    # Sınıflandırma Kısmı
25    Flatten(),
26    Dense(512, activation='relu'), # Büyük Dense Katman
27    Dropout(0.5), # %50 Unutma (Classification kısmında daha agresif)
28    Dense(1, activation='sigmoid')
29 ])
30
31 # Derleme (Adam)
32 model_4.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

3. Sonuç ve Değerlendirme

Bu çalışma, CNN mimarileri kullanılarak yüzey çatlak tespitinin çok yüksek başarıyla yapılabileceğini göstermiştir.

1. Donanım Etkisi: RTX 3060 sayesinde modeller çok hızlı eğitilmiş, özellikle batch size artırıldığında GPU verimliliği maksimize edilmiştir.
2. En İyi Mimari: Senaryo 4 (Dropoutlu Model), hatayı (Loss) minimize etmesi ve neredeyse %100'e yakın başarısı ile en güvenilir model seçilmiştir. Gerçek hayatta bir otonom robot veya drone üzerine entegre edilecekse bu model tercih edilmelidir.
3. Veri Seti Yorumu: Tüm modellerin %99 üzerinde performans vermesi, veri setindeki görüntülerin gürültüsüz ve ayırt edici özelliklere sahip olduğunu göstermektedir. Daha zorlu koşullar (farklı ışık, gölgeler, ıslak zemin vb.) için veri artırma (Data Augmentation) teknikleri ile modelin zorlanması önerilebilir.

4. Kaynak

- <https://www.kaggle.com/datasets/arunrk7/surface-crack-detection>
- <https://alitunacanonar.medium.com/derin-%C3%B6%C4%9Frenme-y%C3%B6ntemleri-ve-uygulamalar%C4%B1-1ce215de40e8>